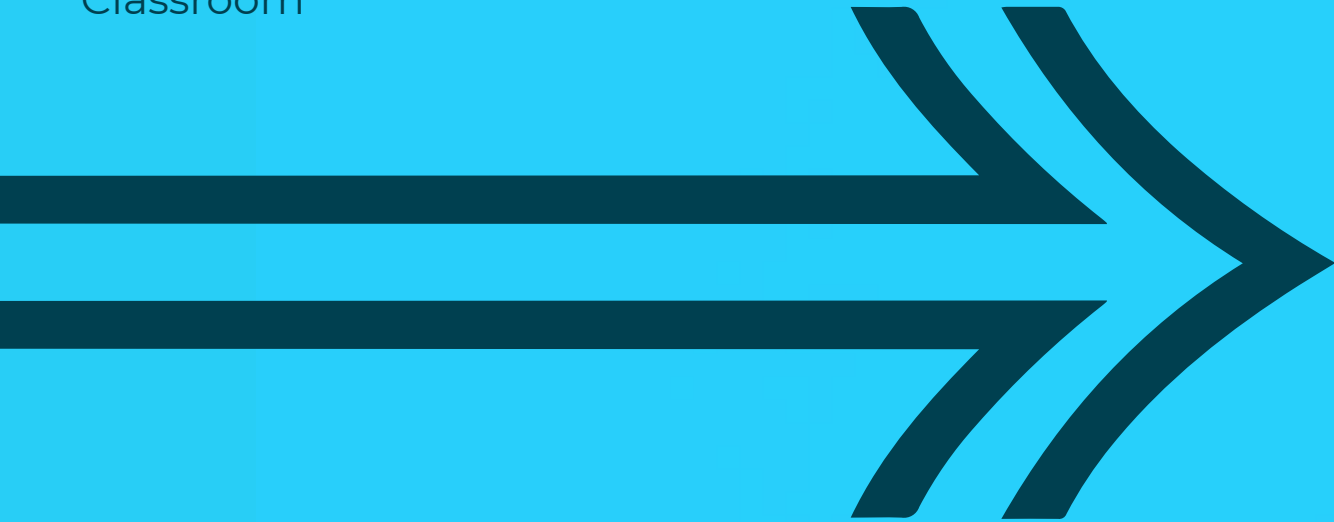




MICROSERVICES PERSISTENT

Classroom





DATA STORE



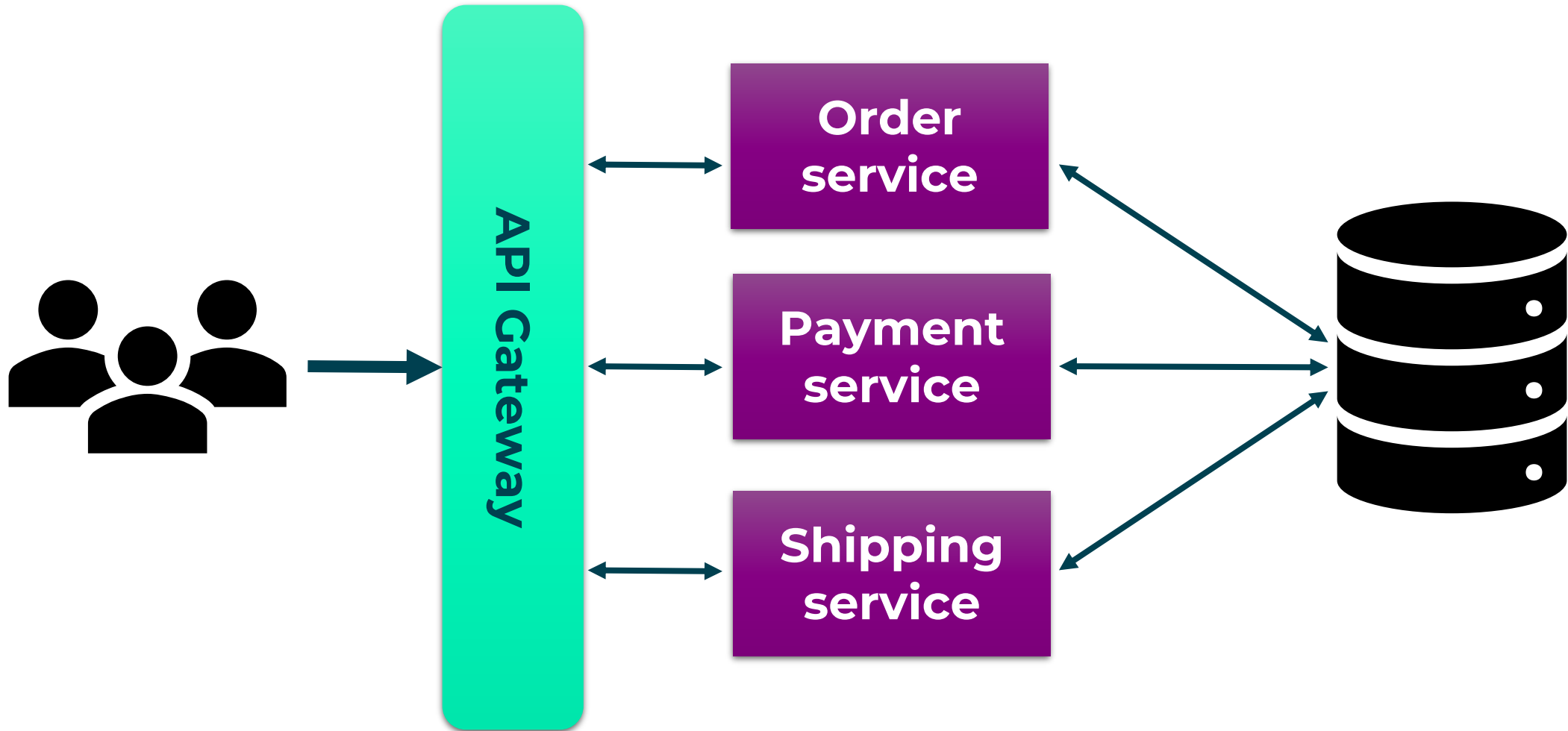
Breaking the Monolithic Database in a Microservices Architecture

Polyglot Persistence

- **Microservices combine components into a single service that provide a single business functionality**
- **Microservices are autonomous, they are self-contained units**
- **Map microservices to agile teams** – A microservice is owned by a single team
 - A team that has ownership and controls its destiny:

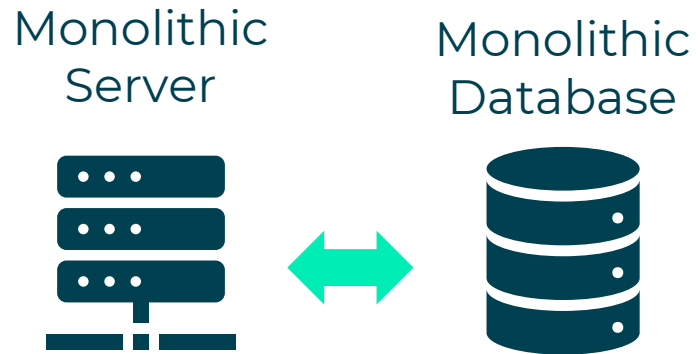
Polyglot: *knowing, using, or written in more than one language*

Classic data store example

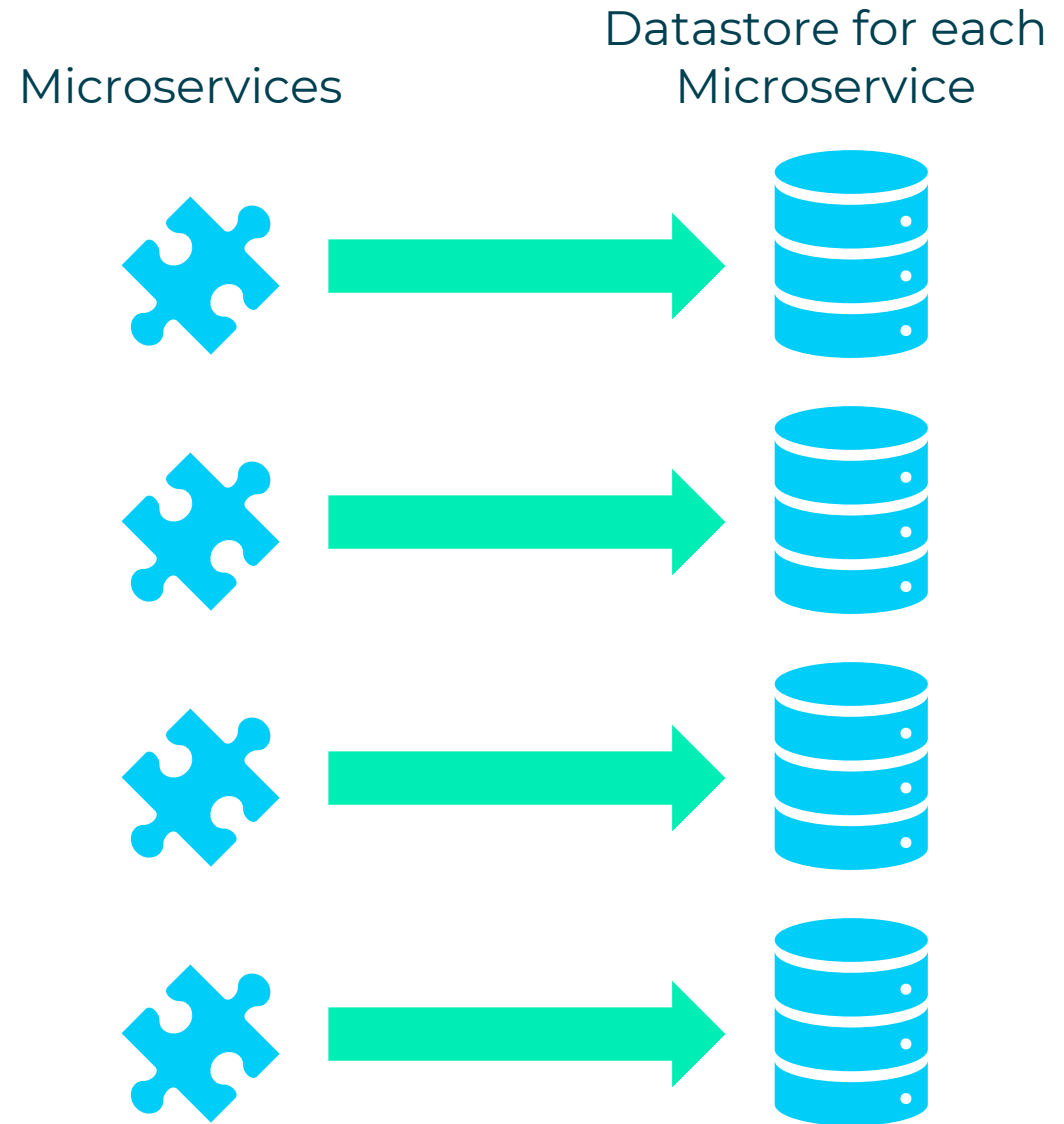


What sort of challenges such a system could pose?

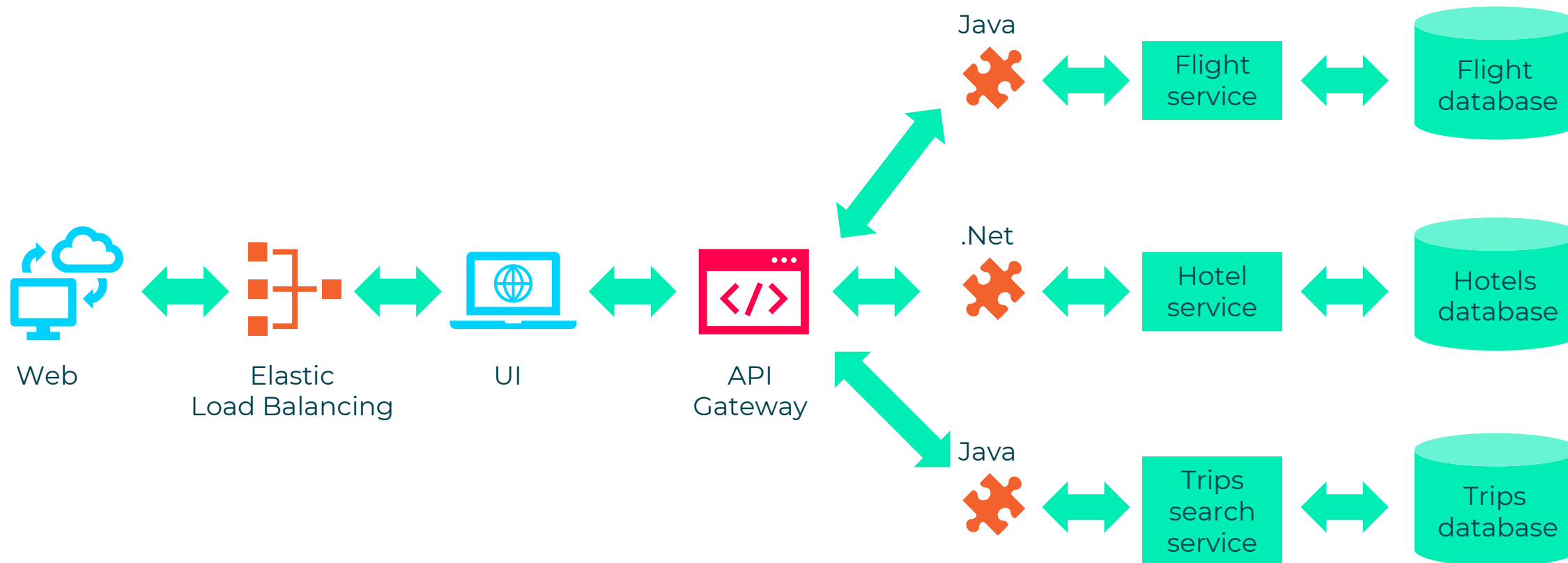
Polyglot Persistence



Journey from Monolith to Microservice



Polyglot – Database example



Polyglot Cont. NoSQL

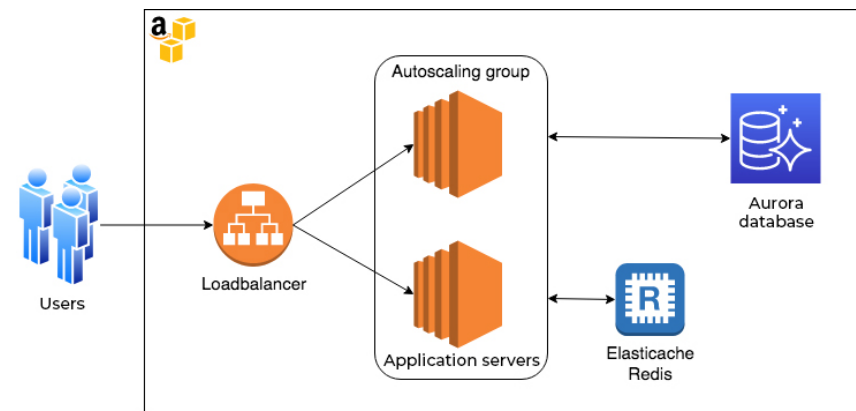
SQL	NoSQL
Optimized for <u>storage</u>	Optimized for <u>compute</u>
Normalized/relational	Denormalized/hierarchical
Ad hoc queries	Instantiated views
Scale vertically	Scale horizontally
Good for online analytical processing (OLAP)	Build for online transaction processing (OLTP) <u>at scale</u>

QA Polyglot – NoSQL (AWS)



**Amazon
Elasticache**

Caching service using the
Memcached and Redis

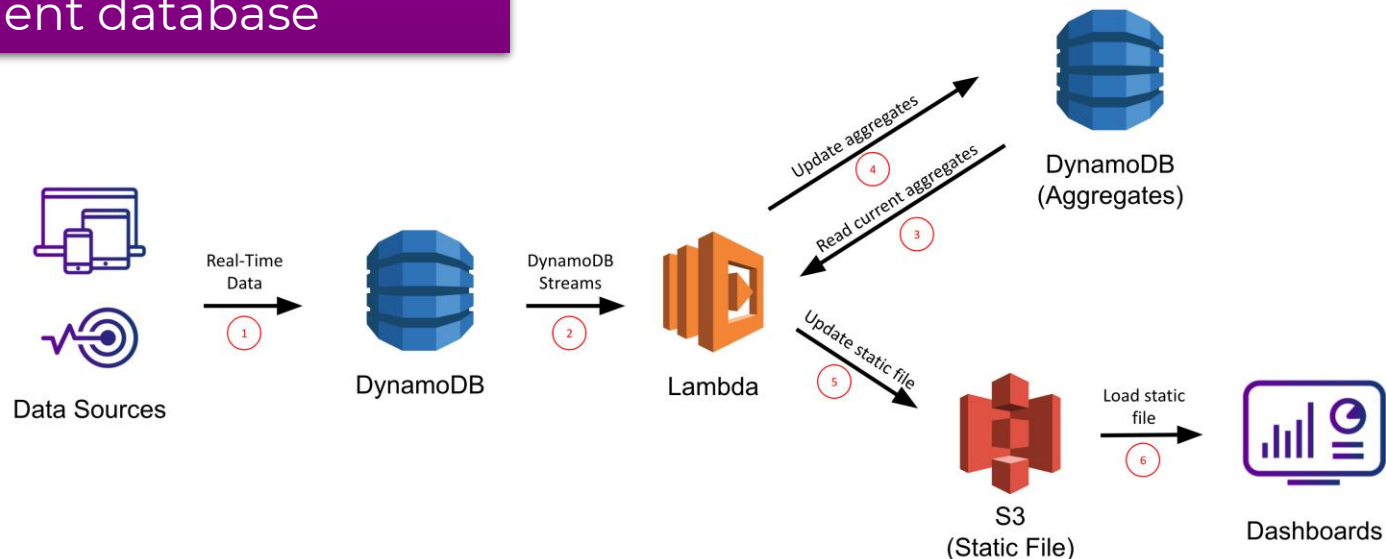


DocumentDB

A MongoDB compatible
document database

DynamoDB

A serverless AWS
proprietary Key/Value &
Document database





POLYGLOT – NOSQL (AWS)



**Amazon
Quantum
Ledger
Database**

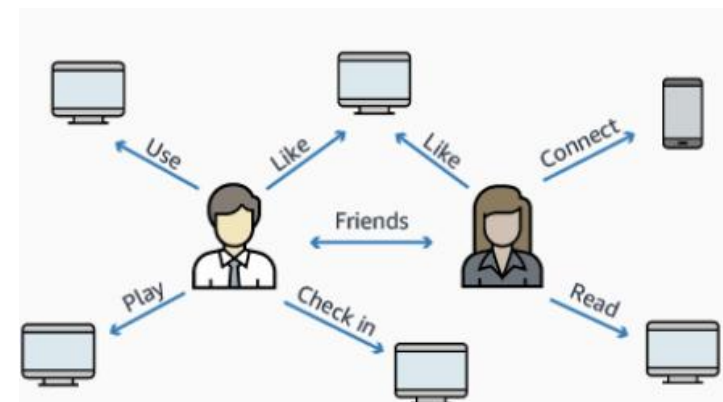
QLdb

Immutable, append only,
cryptographically verifiable
database

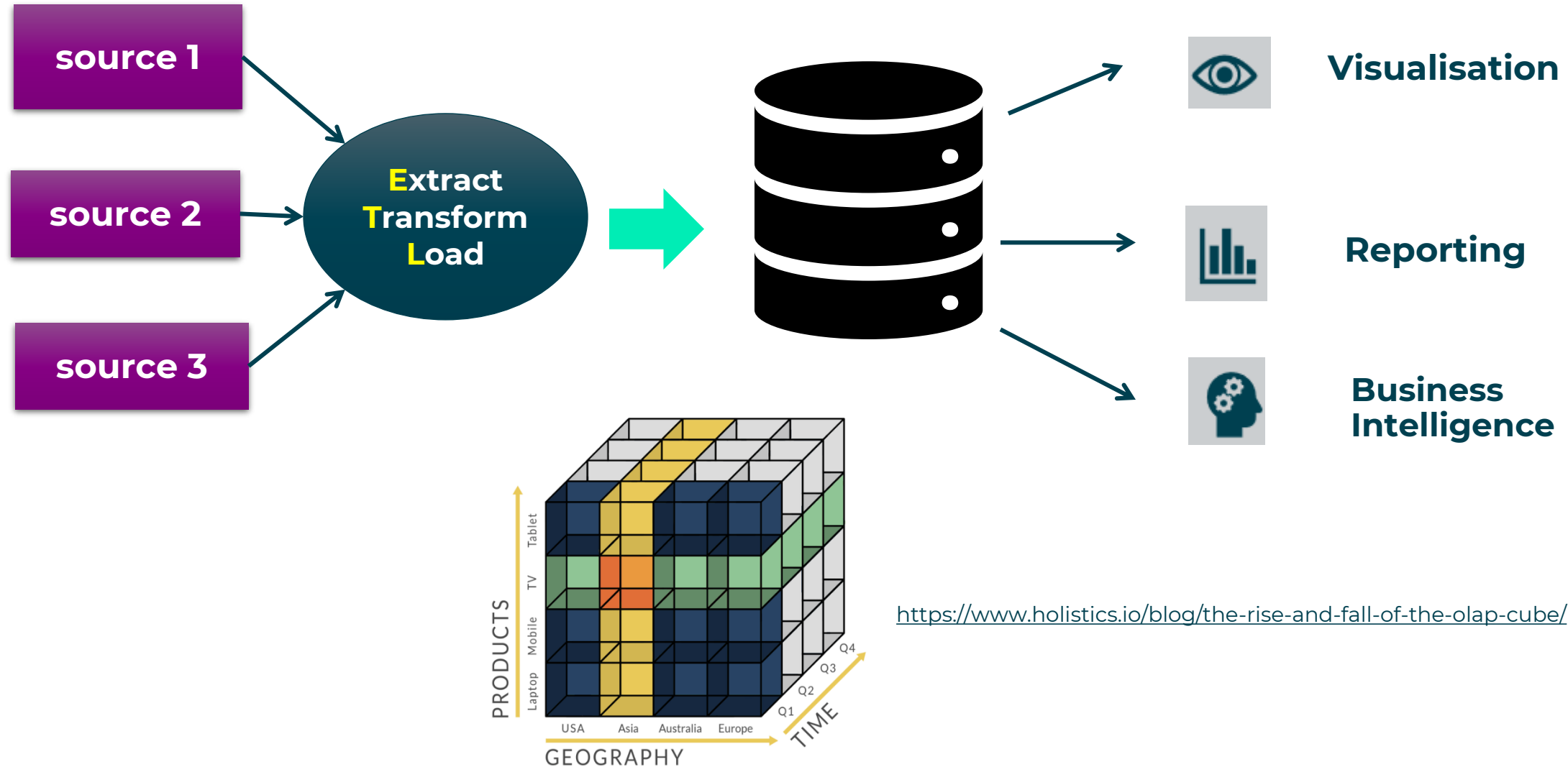


**Amazon
Neptune**

Graph type database when the
many-to-many relationship is as
important as the data itself



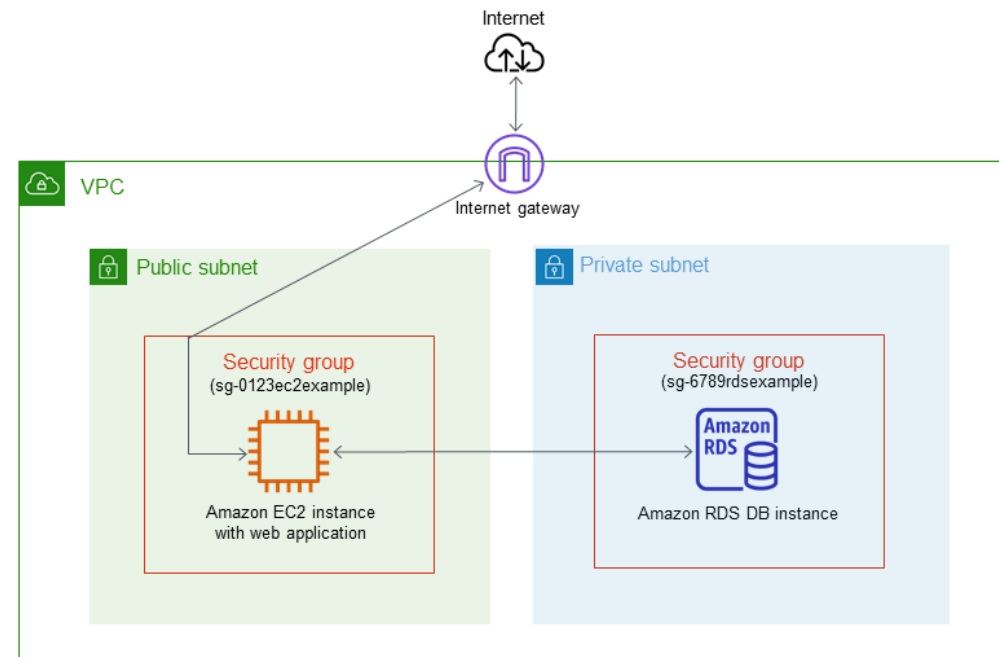
Data warehousing and OLAP



QA Lab activity - RDS and DynamoDB

Upon completion of this Lab, you will be able to:

- Create RDS and DynamoDB databases
- Use the AWS Console to create subnet groups
- Manage access by using security group rules
- Connect to and use the databases you create



RDBMS - RDS



NoSQL - DynamoDB



